UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

Lesson 9: Neural Networks

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

20 November 2026

Agenda

- A boundary whose **shape** is learned, not chosen in advance
- One neuron is one line, and exactly where that runs out
- Forward propagation, and backpropagation derived right to left
- Softmax, activations, and the vanishing gradient
- Initialisation, optimisers, regularisation: the failure modes

Exercise 8, before we start

- What counts is **what the metric could see**, not the score itself
- The gap to watch for: a silhouette reported as endorsement of a clustering, when it can only endorse roundness
- Today the same obligation returns in a new form: a network will reach a high training accuracy very quickly, and that number on its own says almost nothing

What a network adds

- Boundaries so far had a shape **fixed in advance**
- A line; axis-parallel boxes; a curve from a chosen kernel
- A network learns the **shape of the boundary itself**
- Everything else today makes that possible, or stops it going wrong
- No architecture tour, no hardware, no network call

Three problems, and a ceiling of 0.97

- **Acceptance test**: two axes, one pass/fail verdict
- **Two-channel drift**: the exclusive-or (XOR) function
- **Handwritten digits**: 1,797 images, ten classes, real
- The rig is **wrong on 3% of units**, so every score today is read against a ceiling of **0.97**
- A line scores **0.55** on acceptance and **0.93** on digits

The perceptron, and a unit you already have

- Rosenblatt, 1958: weights, a bias, “output 1 if the sum is positive”
- Swap the threshold for a sigmoid: lesson 4’s logistic unit, unchanged
- What is new is one word: **component**
- Undecided at 0.5 means the weighted sum is zero
- A line in two dimensions, a hyperplane in general

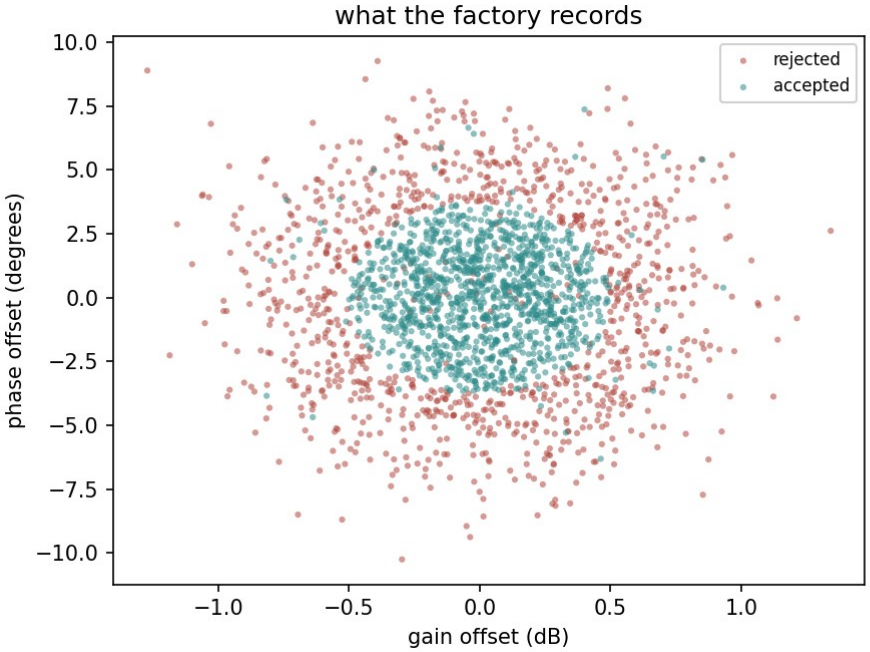
A neuron is a line

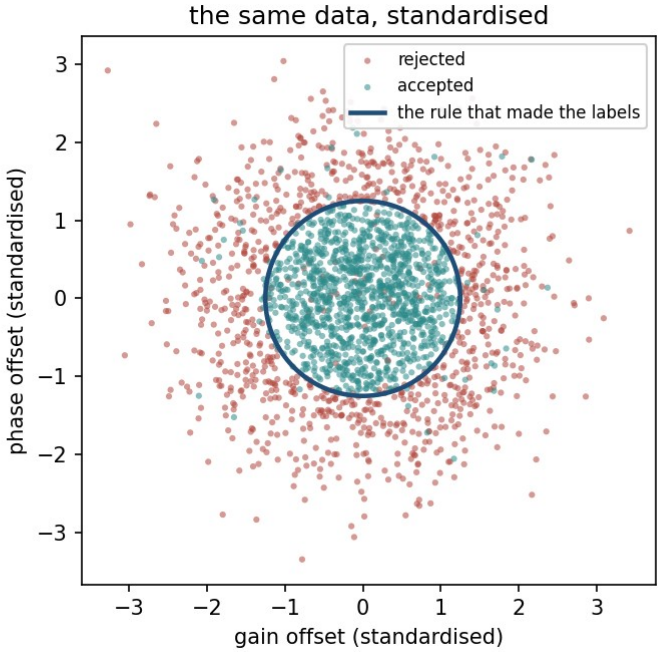
$$\hat{y} = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Meridian’s acceptance test

- Two calibration axes: a gain offset in decibels, a phase offset in degrees
- It passes when both offsets are **jointly** small enough for one global firmware correction
- 2,250 training sensors, 3,000 generated in all
- So the accept region is the inside of a **closed curve**
- And no straight line encloses anything

2,250 sensors, raw and standardised





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

A circle of radius 1.25

- Gain tolerance **0.50**, production spread **0.40**
- Phase tolerance **3.75**, production spread **3.00**
- Both are exactly **1.25 spreads**, so standardised, the accept region is a circle of radius 1.25
- Two units, two tolerances, one shared shape
- **0.5497** of the 3,000 generated units fall inside it

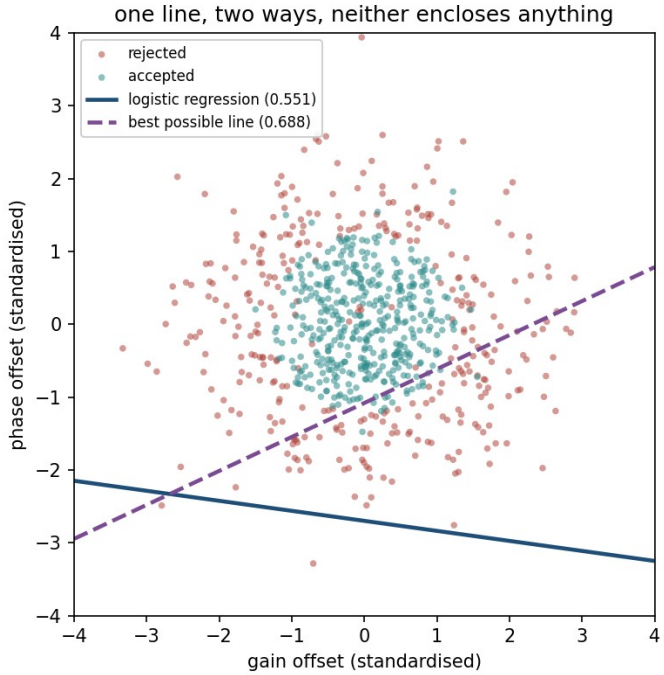
What fraction of units the circle holds

$$P(\|z\| < 1.25) = 1 - e^{-1.25^2/2} = 1 - e^{-0.78125} = 0.5422$$

What one line is worth

	test accuracy
always predict “accepted”	0.5480
fitted logistic regression	**0.5507**
the best straight line for this population	0.6491
the rig’s ceiling	0.9700

No line encloses it



The fit did not fail

- Coefficients **(0.0096, 0.0701)**, intercept **0.1892**
- Almost exactly the constant model, and the correct answer to the question a line can ask
- Not an optimiser that gave up. The reason is symmetry
- The accept region is a disc on the origin; every accepted unit has a partner opposite
- Any tilt gaining accuracy on one side gives it back on the other

72,762 candidate lines, and 4 points of self-deception

- The search tries 72,762 boundaries and scores the winner **on the data that chose it**: 0.6880
- The honest figure for the population, with no test set: **0.6491**
- **Almost 4 points of pure selection bias**
- Nothing was fitted. No parameter estimated. Only a choice made
- Lesson 5’s argument, inside a lesson-9 experiment

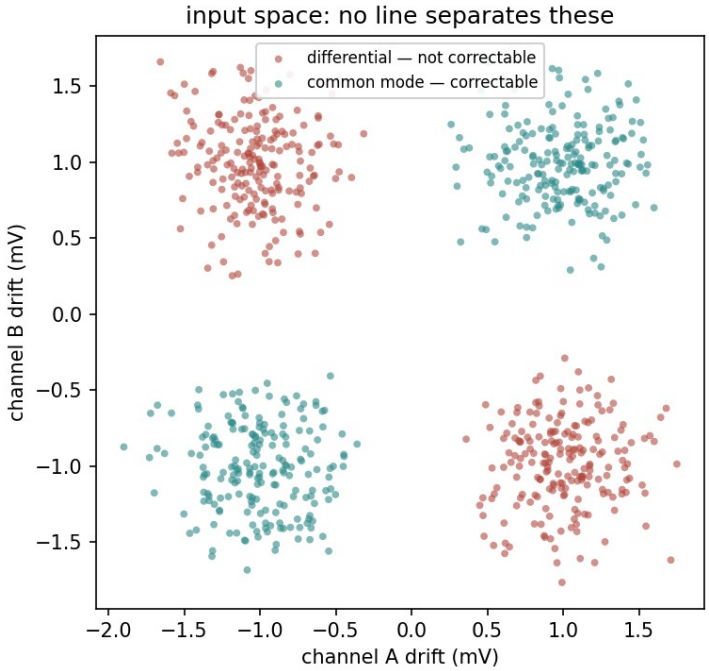
The smallest problem that needs a hidden layer

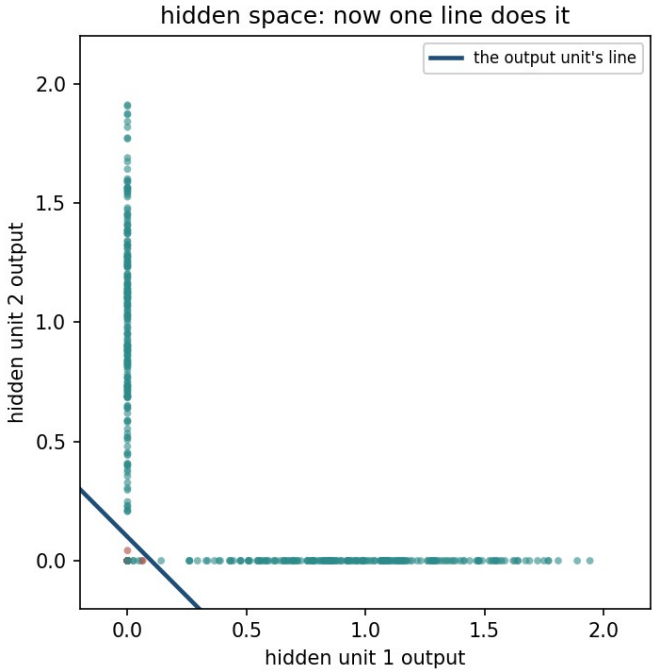
- Removable only when the two drifts **share a sign**; opposite drifts are not
- That is the exclusive-or (XOR) function
- Four clouds of 200 units; a line scores exactly **0.5000**
- Coding the channels as ±1, the rule is about $|a + b|$
- “Large in absolute value” is the outside of a strip: **two boundaries, not one**

Two units, written down rather than trained

- Two rectified linear units (ReLU), each max(0, ·) of the same sum
- One fires when a + b > 1, the other when a + b < -1
- Between them they compute |a + b|, clipped
- Output weights 10 and 10, bias -1: the rule is |a + b| > 1.1
- **Nothing trained**, and over all 800 units it scores **0.9938**

What the hidden layer actually did



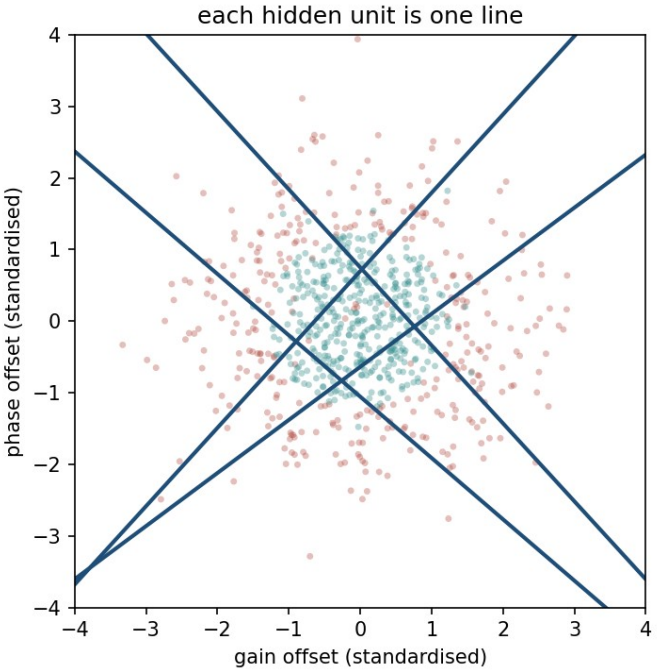


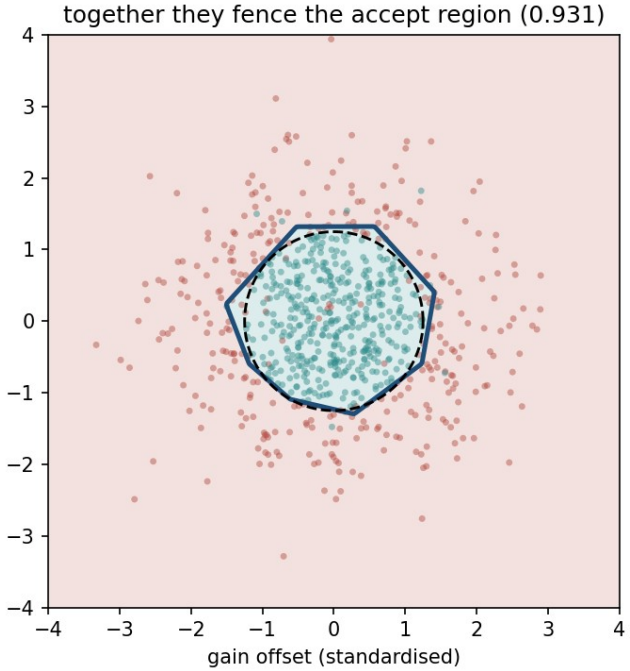
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The hidden layer classified nothing

- It **moved the data** until the output line was enough
- Correctable clouds to (1,0) and (0,1); the rest to the origin
- The name for this is **representation learning**
- What is learned: coordinates in which the boundary is simple
- Lesson 10’s convolutional networks: the same trick, constrained

Four lines enclose a region; one never does

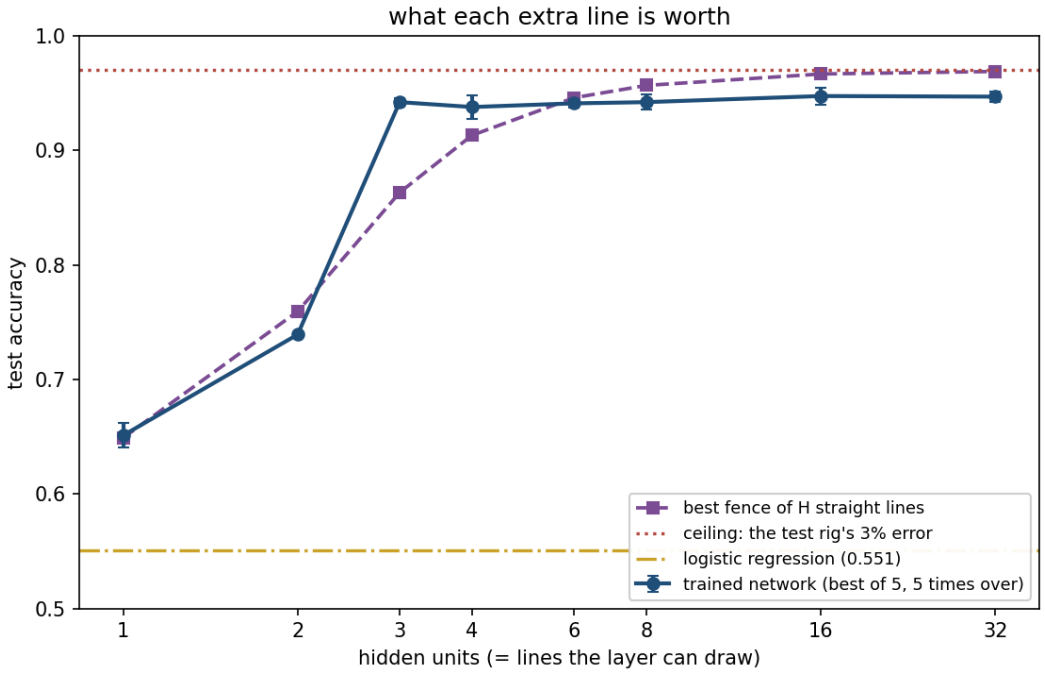




Capacity is not the same as findability

hidden units	median accuracy	best of 20	runs above 0.95
2	0.7500	0.9975	**4 of 20**
3	1.0000	1.0000	15 of 20
4	1.0000	1.0000	19 of 20
8	1.0000	1.0000	20 of 20

What each extra line is actually worth



Universal approximation, and what it does not promise

- Cybenko (1989), Hornik (1991): one hidden layer, enough units, any continuous function
- Read the quantifiers. It says such a network **exists**
- Nothing about how many units “enough” is
- Nothing about whether training will find it
- Nothing about behaviour outside the region
- Two units sufficed on the drift data: descent missed it 16 times in 20

Forward propagation: examples in rows

- One hidden layer of H units, one output unit, m examples, n inputs
- Two matrix multiplies, each followed by an elementwise function
- This course puts examples in **rows**: as scikit-learn and Keras do
- Textbooks often use columns, and every transpose flips if you do

Layer 1: a line per unit, then a non-linearity

$$Z^{[1]} = XW^{[1]} + b^{[1]}, \qquad A^{[1]} = g(Z^{[1]})$$

Layer 2: one more line, then a probability

$$Z^{[2]} = A^{[1]} W^{[2]} + b^{[2]}, \qquad \hat{y} = \sigma(Z^{[2]})$$

The shapes

symbol	shape	what it is
X	$m \times n$	the design matrix, one example per row
W, layer 1	$n \times H$	one column per hidden unit
b, layer 1	H	broadcast across rows
Z and A, layer 1	$m \times H$	pre-activations and activations
W, layer 2	$H \times 1$	the output unit’s weights
$\hat{y}$	$m \times 1$	one prediction per example

Two facts the shapes hand you

- **Examples never mix**: the row index passes through untouched
- Nothing lets example 3 influence example 7, which is what makes mini-batching valid
- **Each column of the first weight matrix is one unit**
- Its weight vector, and the line that unit draws
- So H hidden units are H lines, and the output unit votes

An untrained network costs 0.6721; guessing costs 0.6931

$$J = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

Break

- Twelve minutes

Two bad ways to get every gradient

- We need the derivative of the cost for **every** weight and bias
- **Perturb and re-run**: one forward pass per parameter, 301,066 of them, for one step
- **Differentiate symbolically**: the same sub-expressions, thousands of times
- Both correct. Both unusable

Compute each repeated piece once, right to left

- Carry the derivative of the cost with respect to a layer's **pre-activations**
- Given it for one layer: that layer's weight gradients, **and** the layer below
- One backward sweep produces every gradient in the network
- **A weight's gradient is how wrong the layer above was, times what this weight contributed**

The output layer collapses to prediction minus truth

$$\delta^{[2]} = \frac{1}{m}(\hat{y} - y)$$

Why sigmoid and cross-entropy belong together

- The loss derivative carries a factor 1 over $\hat{y}(1 - \hat{y})$
- The sigmoid's derivative **is** $\hat{y}(1 - \hat{y})$
- They cancel exactly, leaving $\boldsymbol{\hat{y} - y}$
- Squared error keeps that factor, and it is near zero exactly when the network is **confidently wrong**

One step down, through the activation

- Weight gradient = incoming activations, transposed, times the signal above
- To step down: multiply by the weight matrix above, then by the activation’s derivative
- For the ReLU that derivative is an indicator: 1 where the pre-activation was positive
- A unit that contributed nothing forward passes nothing backward

Backward through one layer

$$\delta^{[1]} = (\delta^{[2]}(W^{[2]})^\top) \odot g'(Z^{[1]})$$

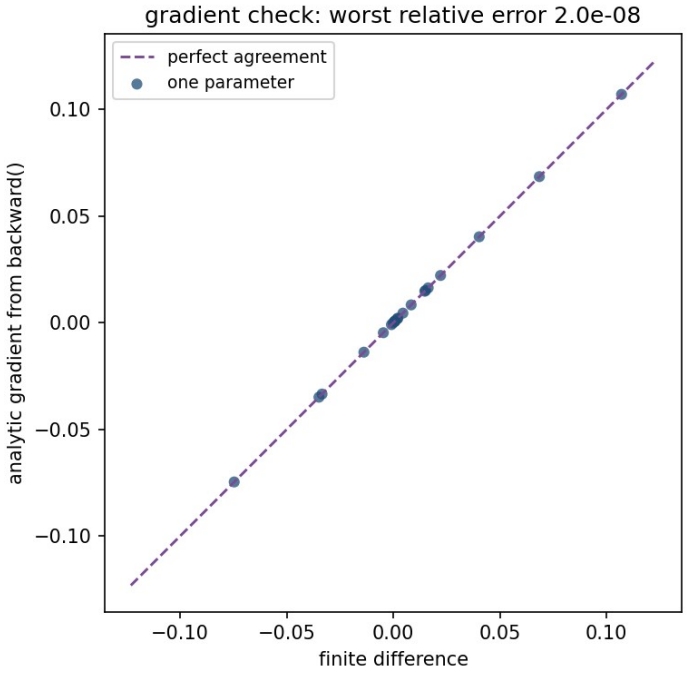
What backpropagation costs

- The backward pass does about the same arithmetic as the forward pass
- **A gradient costs roughly what a prediction costs**: this is what makes training feasible at all
- But every layer’s activations must be **stored**
- Memory grows with depth times batch size, and memory, not arithmetic, is what usually limits batch size

Check the gradient before you trust it

- A wrong gradient **raises no exception**: it trains badly and looks like a modelling problem
- Compare each partial against a central difference: error of order h^2
- Notebook 01: worst disagreement 1.97×10^{-8}, median 7.83×10^{-11}
- **Below 10^{-6} believe it; above 10^{-4} there is a bug**
- Four lines of code, and not optional

21 partial derivatives, on the diagonal



Notebook 1, live

- Backpropagation from scratch, gradient-checked before it is trusted
- The two-unit hand-built network, then the same problem trained from 20 random starts at each width
- The polygon yardstick: how well could H lines fence a circle?
- The one initialisation that cannot work, and the cost it converges to

Ten classes need ten outputs

- **Softmax** turns K scores into a probability distribution
- For K = 2 it reduces to the sigmoid: one construction, two sizes
- The loss is categorical cross-entropy: minus the log probability given to the true class
- The gradient at the output is again $\hat{\mathbf{y}} - \mathbf{y}$, so everything derived before the break applies unchanged

Softmax

$$\operatorname{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

A hidden layer is not always the answer

architecture	parameters	validation accuracy	sd
softmax alone, no hidden layer	650	0.9324	0.0035
one hidden layer of 32	2,410	0.9602	0.0052
one hidden layer of 64	4,810	0.9611	0.0045
two hidden layers of 64	8,970	0.9676	0.0057

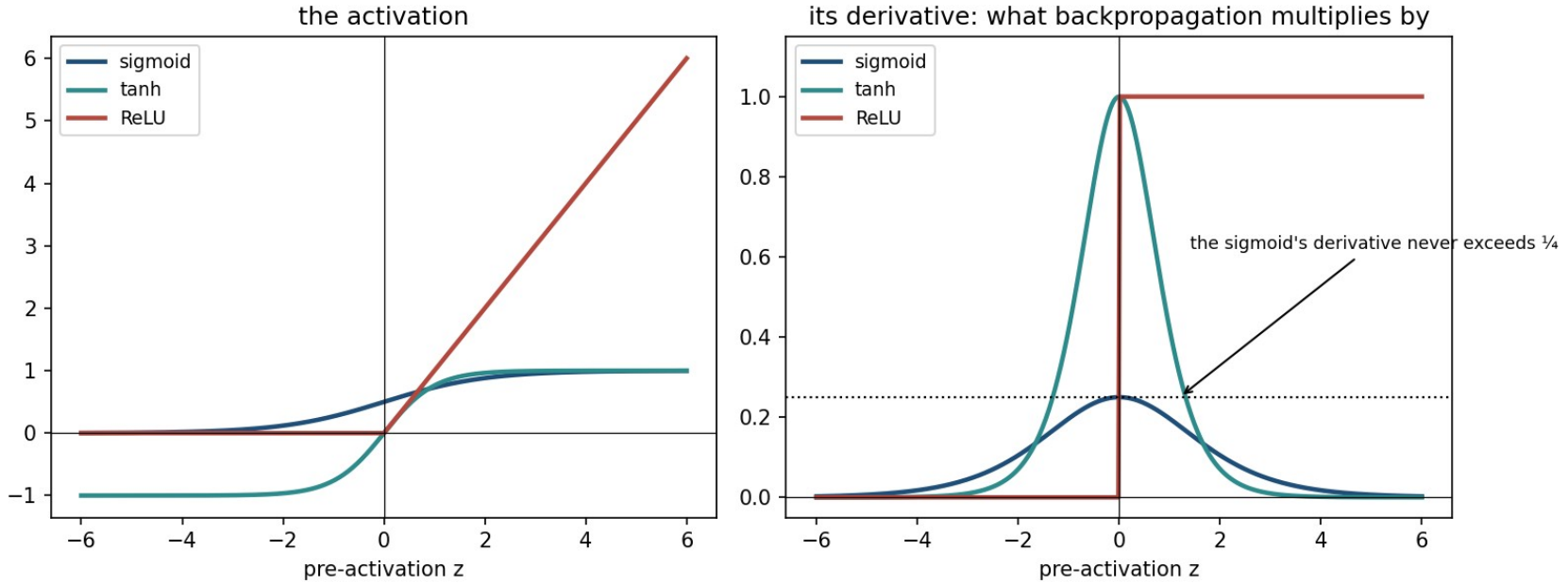
Reading that table

- No hidden layer at all is within **3.5 points** of the best network here
- The first hidden layer is worth **2.8 points**; 32 units to 64, **0.09**
- On the acceptance data the same step was worth **39 points**
- Digits vote pixel by pixel: the acceptance rule needed two measurements **at once**

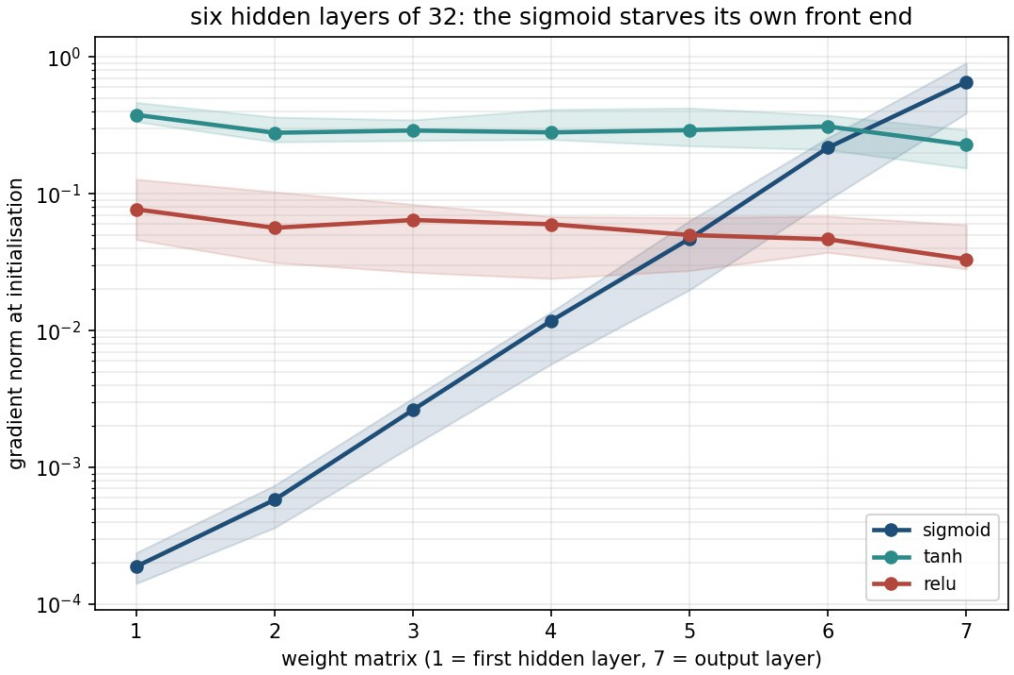
A sigmoid layer divides the gradient by about four

- Any number of linear layers composes to one: the activation is the whole reason depth buys anything
- The sigmoid’s derivative $\sigma(z)(1 - \sigma(z))$ peaks at **¼**, at zero only
- Backpropagation multiplies by it **once per layer**
- Initialisation leaves the weight factor near 1, so the derivative decides: **each sigmoid layer divides the gradient by about four**

Three activations, and their derivatives



Six layers of it, measured



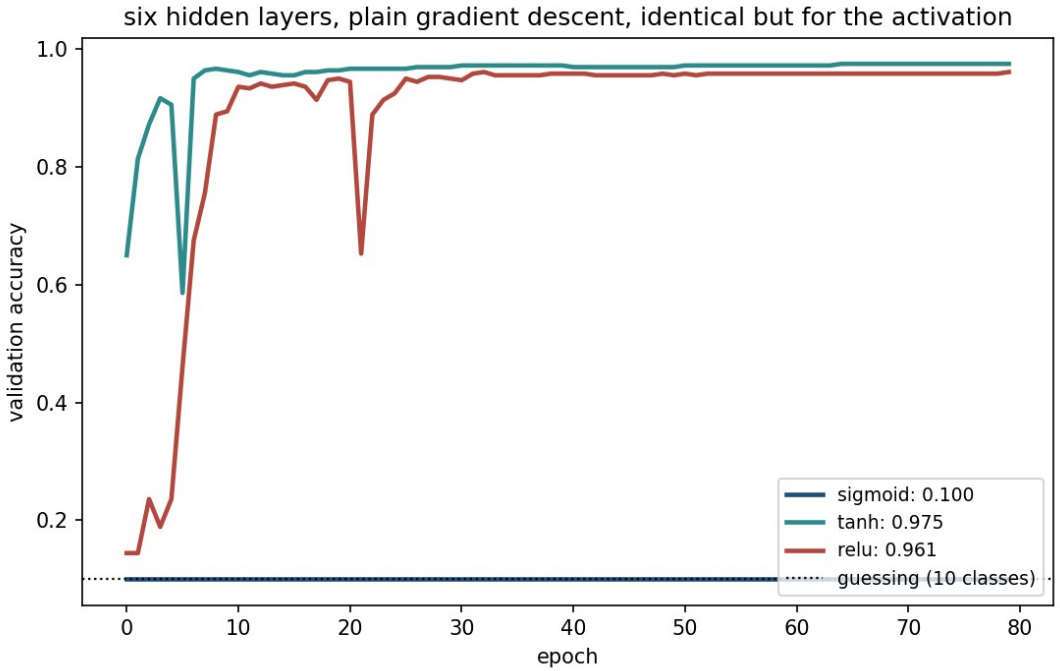
One draw is not a law

- End-to-end ratio, last weight matrix to first: median **3,547**
- Over the same eight seeds it ranges **2,734 to 4,607**
- Quoting the largest reports one draw as a law
- **The per-layer factor of about 4 is the property**
- By depth: 9.5, 170, 3,553, 46,250 at two, four, six and eight layers

Eighty epochs: only one of them never learns

- Same six layers, same data, differing **only** in the activation
- Sigmoid: **0.1000**, exact chance on ten classes
- tanh: **0.9750**, past 0.85 within 3 epochs
- Rectified linear unit: **0.9611**, 9 epochs to get going
- What kills the sigmoid is not squashing: it is *where its derivative is bounded*

The sigmoid never leaves chance



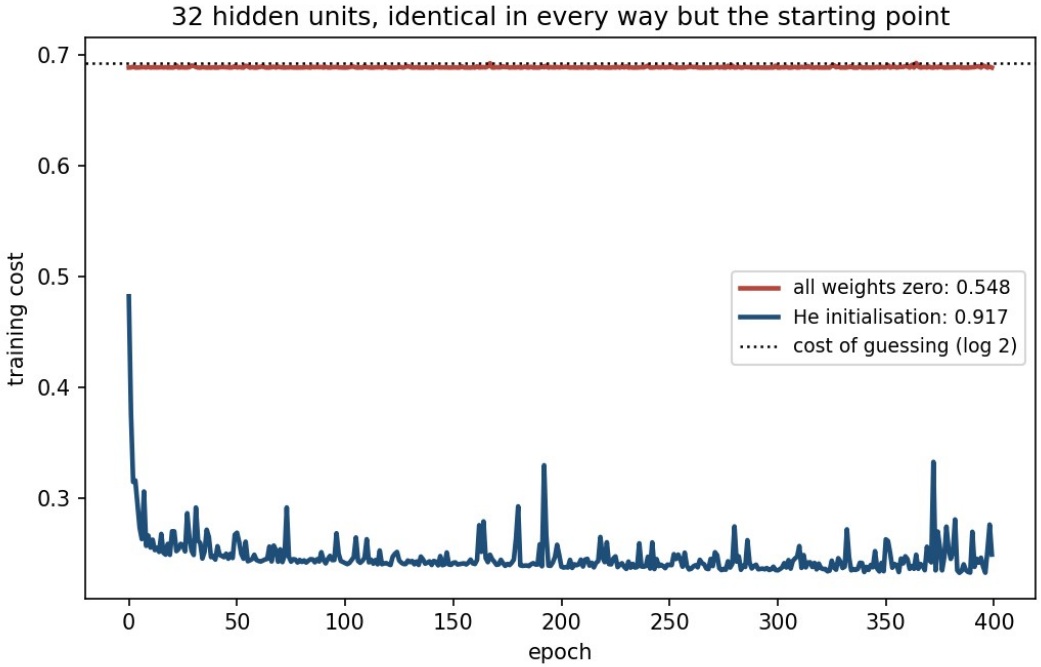
Notebook 2, live

- The same network again, this time in six lines of Keras, and the same answer
- Softmax on ten classes, and the depth comparison behind the table
- The per-layer gradient shrinkage, measured across eight seeds at five depths
- How big the random weights should be, and what happens at both extremes

Zero cannot work

- Lesson 3 started from zero, and was right to: those costs are convex
- All-zero weights **freeze** a network: every gradient is exactly zero
- **Only the output bias ever moves**
- The bias converges to the base rate, so the cost converges to the label entropy: **0.6887 predicted, 0.6887 measured**
- Distinct hidden columns: **1 of 32**

Only the starting point differs



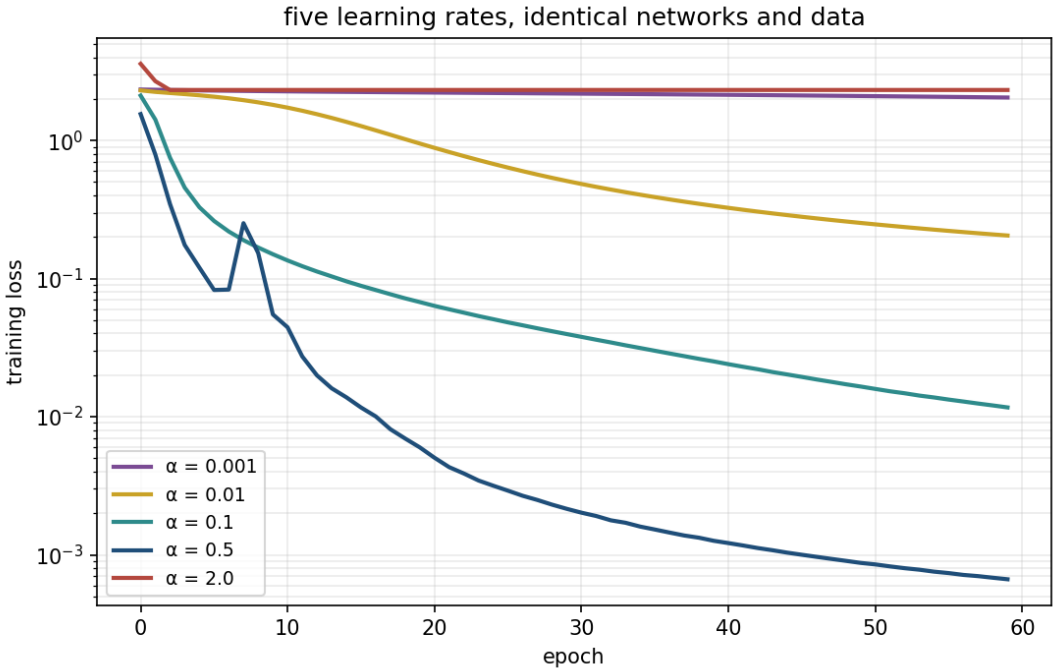
How large? Propagate the variance

- A unit sums n inputs, so variance is multiplied by $n \times \mathrm{Var}(w)$ each layer
- Set $\mathrm{Var}(w) = 1/n$: **Glorot**. ReLU halves it, so it wants $2/n$: **He**
- **Too small collapses**: indistinguishable from zero by layer 4
- **Too large saturates** rather than exploding: 73% of the last layer past 0.99
- Glorot loses half its spread over eight layers

The learning rate, swept

α	final training loss	validation accuracy	sd
0.001	1.9550	0.5574	0.0794
0.01	0.1965	0.9398	0.0035
0.1	0.0115	0.9556	0.0039
0.5	0.0007	**0.9685**	0.0026
2.0	2.3222	0.1009	0.0013

Three regimes, and a narrow band between them



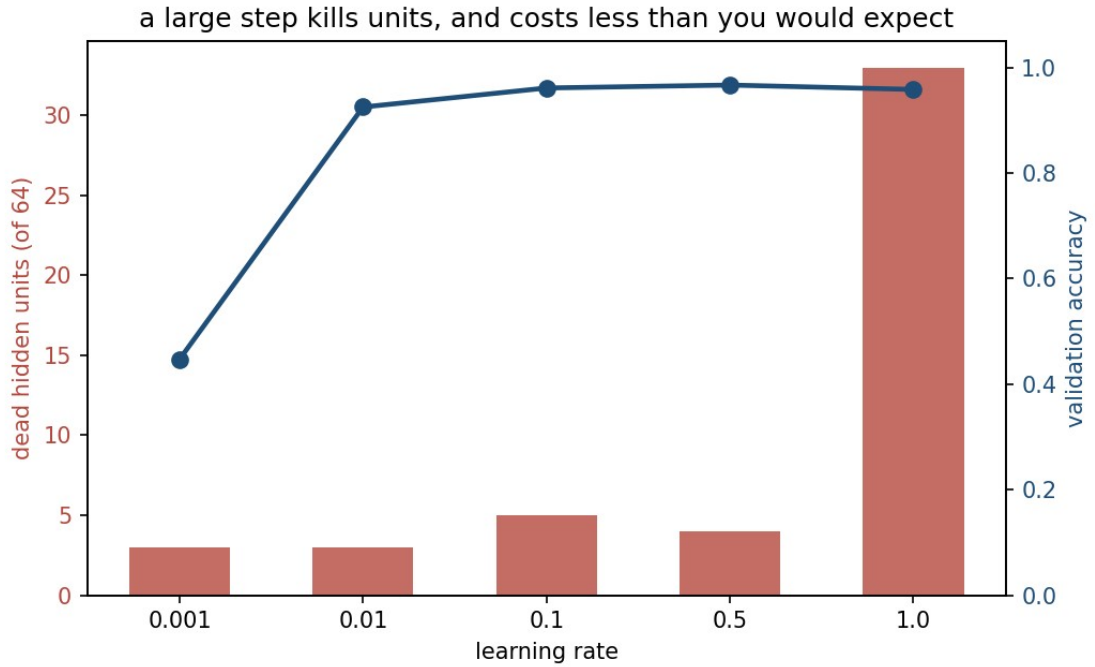
The predictable mistake

- Too small, and the loss is still falling when the epochs run out
- **It looks exactly like a network that is too small**: both accuracies low
- The instinct that follows is sound: low training accuracy really is the signature of underfitting
- **Vary α over orders of magnitude before touching the architecture**

The rectified linear unit has its own failure

- Its derivative is exactly 1 on the active side: so, no vanishing gradient
- But a unit negative for **every** example gets zero gradient, for ever
- **Dead**: the gradient that would revive it is the one it cannot receive
- At $\alpha = 1.0$, **33 of 64 are dead**: accuracy 0.9583 against a best of 0.9667

Half the layer dead, and almost no visible cost



Mini-batches, momentum, and Adam

- Mini-batch **stochastic gradient descent (SGD)**: noisy, unbiased, vectorises
- **Momentum** averages past gradients: oscillations cancel, and lesson 3’s stretched valley is repaired
- **Adam (adaptive moment estimation)**: a per-parameter step size
- Which is what makes it forgiving of a badly chosen global α

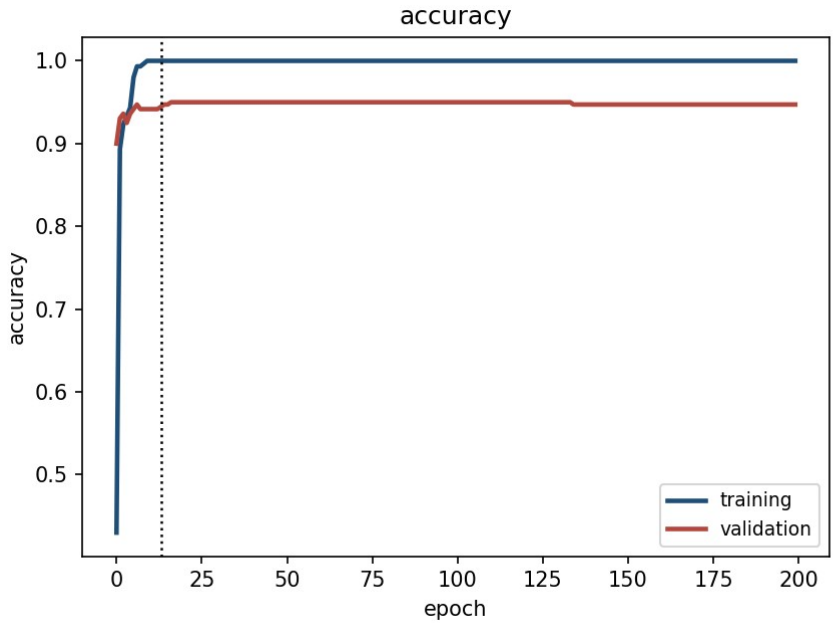
Adam, measured: read the spread before the mean

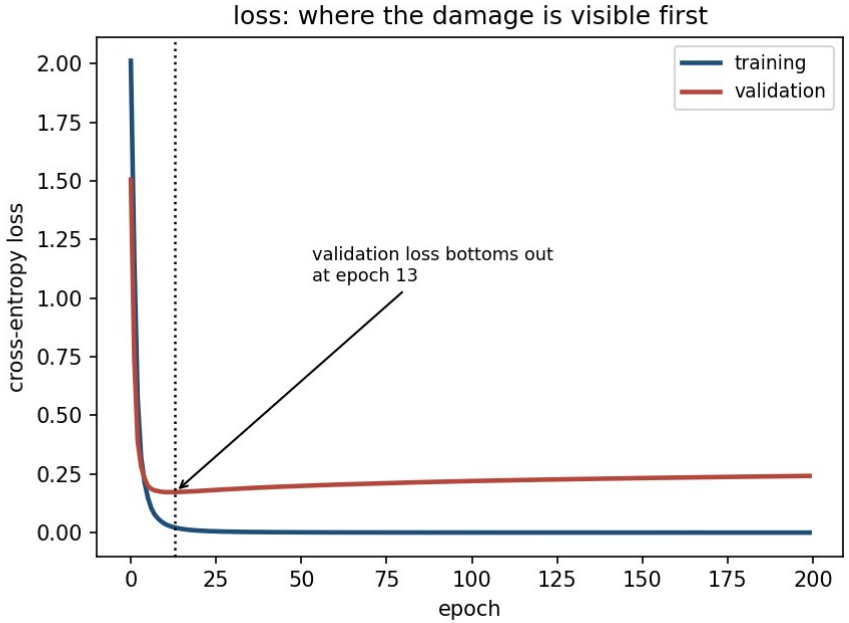
optimiser	test accuracy	sd	worst of 5	vs the true rule
plain gradient descent	0.9389	0.0184	0.9027	0.9685
with momentum 0.9	0.9349	0.0094	0.9173	0.9645
Adam	**0.9485**	0.0072	0.9360	0.9792

Three regularisers, and what they bought

method	test accuracy	sd	train − test gap	epochs run
early stopping only	0.9411	0.0069	0.0589	38.8
+ dropout 0.4	0.9483	0.0038	0.0503	48.8
+ L2 10^{-3}	0.9478	0.0011	0.0522	198.4
+ dropout and L2	**0.9511**	0.0045	0.0489	161.6

It memorises the training set, and generalises anyway





Notebook 3, live

- The learning-rate sweep and the dead-unit count, from scratch
- Plain descent, momentum and Adam from an identical start
- **301,066 parameters on 300 examples**: memorises, generalises anyway
- Validation *loss* climbs while *accuracy* stays flat
- More data beats more tuning, three to one

What to take away

- **A neuron is a line**: three enclose a region, one never does
- A hidden layer **moves the data**; it does not classify
- **Backpropagation is the chain rule** - so gradient-check it
- **A sigmoid layer divides the gradient by about four**
- Report a difference with its spread

Homework

- **Exercise 9**, discussed at the start of **Friday 27 November**
- `Exercises/09_neural_networks.md`